



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Plataforma web para la
descarga centralizada de
software con gestión de
bundles y búsqueda semántica
mediante LLMs**



Presentado por José Gallardo Caballero
en Universidad de Burgos — 25 de junio
de 2026

Tutores: José Manuel Aroca Fernández,
Rodrigo Pascual García



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. José Manuel Aroca Fernández, profesor del Departamento de Ingeniería Informática de la Universidad de Burgos,

Expone:

Que el alumno D. José Gallardo Caballero, con DNI 71478806G, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado Plataforma web para la descarga centralizada de software con gestión de bundles y búsqueda semántica mediante LLMs.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 25 de junio de 2026

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

D. José Manuel Aroca Fernández

D. Rodrigo Pascual García

Resumen

La instalación inicial de un equipo obliga habitualmente a localizar y descargar cada aplicación desde una página distinta. Este Trabajo de Fin de Grado plantea el diseño de *Batch Downloader*, una plataforma web que centralizará un catálogo de software y permitirá seleccionar varias aplicaciones para obtener sus instaladores oficiales agrupados en un único archivo ZIP.

La plataforma no mantendrá ejecutables almacenados de forma permanente. Cada solicitud dará lugar a un trabajo asíncrono que resolverá los enlaces oficiales, validará los destinos, descargará temporalmente los archivos, generará el ZIP y eliminará los artefactos cuando termine o expire el proceso. El sistema incorporará cuentas de usuario, bundles públicos y privados, una valoración binaria mediante estrellas, solicitudes de nuevas aplicaciones y un panel de administración para mantener el catálogo y revisar errores.

Como ampliación, se diseña una capa de búsqueda semántica basada en embeddings y un modelo de lenguaje. Esta capa permitirá interpretar consultas expresadas en lenguaje natural, proponer bundles y explicar incidencias, pero no tendrá autoridad para descargar archivos, modificar dominios permitidos ni eludir las validaciones de seguridad.

La arquitectura propuesta utiliza Vite y TypeScript en el cliente, Java 26 y Spring Boot en el servidor, MySQL para los datos transaccionales, PostgreSQL con pgvector para los vectores semánticos, RabbitMQ para el procesamiento asíncrono, Playwright para la resolución dinámica de páginas y contenedores Docker para cada servicio.

Descriptores

descarga de software, bundles, Spring Boot, Playwright, procesamiento asíncrono, búsqueda semántica, embeddings, modelos de lenguaje, seguridad web

Abstract

Setting up a computer usually requires locating and downloading every application from a different website. This Bachelor's Degree Final Project proposes the design of *Batch Downloader*, a web platform that will provide a centralized software catalogue and allow users to select several applications and obtain their official installers grouped into a single ZIP archive.

The platform will not store executable files permanently. Each request will create an asynchronous job that resolves official links, validates destinations, temporarily downloads the files, generates the ZIP archive, and removes all temporary artefacts when the process finishes or expires. The system will include user accounts, public and private bundles, binary star ratings, software requests, and an administration panel for catalogue maintenance and error review.

As an extension, the design includes a semantic search layer based on embeddings and a language model. This layer will interpret natural-language queries, suggest bundles, and provide clearer explanations of failures. However, it will not be allowed to download files, change trusted domains, or bypass the platform's security controls.

The proposed architecture uses Vite and TypeScript for the client, Java 26 and Spring Boot for the server, MySQL for transactional data, PostgreSQL with pgvector for semantic vectors, RabbitMQ for asynchronous processing, Playwright for dynamic page resolution, and Docker containers for each service.

Keywords

software download, bundles, Spring Boot, Playwright, asynchronous processing, semantic search, embeddings, large language models, web security

Índice general

Índice general	iii
Índice de figuras	vi
Índice de tablas	vii
Lista de siglas, acrónimos y abreviaturas	viii
1. Introducción	1
1.1. Contexto y problema	1
1.2. Motivación	2
1.3. Alcance de esta memoria	2
1.4. Estructura del documento	3
2. Objetivos del proyecto	4
2.1. Objetivo general	4
2.2. Objetivos funcionales	4
2.3. Objetivos de descarga y procesamiento	5
2.4. Objetivos de búsqueda semántica	6
2.5. Objetivos técnicos y de calidad	6
2.6. Objetivos de seguridad y sostenibilidad	7
3. Conceptos teóricos	8
3.1. Catálogo centralizado y fuentes oficiales	8
3.2. Bundles y personalización temporal	8
3.3. Resolución de enlaces y scraping	9
3.4. Procesamiento asíncrono	9

3.5. Generación temporal del ZIP	10
3.6. Embeddings y búsqueda vectorial	10
3.7. Modelo de lenguaje y generación aumentada con recuperación	11
3.8. Seguridad en la descarga de recursos externos	12
4. Técnicas y herramientas	13
4.1. Criterios de selección	13
4.2. Frontend	13
4.3. Backend	14
4.4. Resolución de fuentes	15
4.5. Persistencia	15
4.6. Mensajería y almacenamiento temporal	16
4.7. Búsqueda semántica y LLM	16
4.8. Contenedores y despliegue	17
4.9. Herramientas de desarrollo y documentación	17
4.10. Resumen de correspondencia	17
5. Aspectos relevantes del desarrollo del proyecto	18
5.1. Enfoque de desarrollo	18
5.2. Actores y permisos	19
5.3. Arquitectura planificada	19
5.4. Flujo general de descarga	20
5.5. Resolución de instaladores	22
5.6. Generación y contenido del ZIP	23
5.7. Bundles y estrellas	23
5.8. Búsqueda semántica y asistencia	25
5.9. Seguridad	27
5.10. Gestión de errores	28
5.11. Observabilidad	29
5.12. Estrategia de pruebas	29
6. Trabajos relacionados	30
6.1. Ninite	30
6.2. Patch My PC Home Updater	30
6.3. UBULabs	31
6.4. Comparación funcional	32
6.5. Diferencias de enfoque	32
6.6. Lecciones aplicadas al diseño	33
7. Conclusiones y líneas de trabajo futuras	34
7.1. Conclusiones de la fase de análisis y diseño	34

<i>Índice general</i>	v
7.2. Limitaciones actuales	35
7.3. Líneas de trabajo futuras	35
7.4. Cierre	36
Bibliografía	38

Índice de figuras

5.1. Secuencia general de generación y descarga de un ZIP.	21
5.2. Secuencia de creación y descarga de un bundle.	24
5.3. Secuencia de búsqueda semántica y explicación mediante LLM.	26

Índice de tablas

4.1. Tecnologías propuestas por responsabilidad.	17
5.1. Permisos generales por actor.	19
6.1. Comparación conceptual de las soluciones relacionadas con Batch Downloader.	32

Lista de siglas, acrónimos y abreviaturas

Esta lista reúne las abreviaturas utilizadas en la memoria y en la documentación técnica, junto con su significado.

Sigla	Significado
AMQP	<i>Advanced Message Queuing Protocol</i> (protocolo avanzado de colas de mensajes)
API	<i>Application Programming Interface</i> (interfaz de programación de aplicaciones)
CAPTCHA	<i>Completely Automated Public Turing test to tell Computers and Humans Apart</i>
CDN	<i>Content Delivery Network</i> (red de distribución de contenidos)
CRUE	CRUE Universidades Españolas; originalmente, Conferencia de Rectores de las Universidades Españolas
CSS	<i>Cascading Style Sheets</i> (hojas de estilo en cascada)
CU	Caso de uso
DNS	<i>Domain Name System</i> (sistema de nombres de dominio)
DTO	<i>Data Transfer Object</i> (objeto de transferencia de datos)
E2E	<i>End to End</i> (extremo a extremo)
ES	<i>ECMAScript</i>
HTML	<i>HyperText Markup Language</i> (lenguaje de marcado de hipertexto)
HTTPS	<i>HyperText Transfer Protocol Secure</i>
IA	<i>Inteligencia artificial</i>
ID	<i>Identifier</i> (identificador)

Continúa en la página siguiente

Lista de siglas, acrónimos y abreviaturas (continuación)	
Sigla	Significado
IP	<i>Internet Protocol</i> (protocolo de Internet)
JPA	<i>Jakarta Persistence API</i>
LLM	<i>Large Language Model</i> (modelo de lenguaje de gran tamaño)
ODS	<i>Objetivos de Desarrollo Sostenible</i>
REST	<i>Representational State Transfer</i> (transferencia de estado representacional)
SOS1–SOS4	Competencias de sostenibilidad 1 a 4
SSRF	<i>Server-Side Request Forgery</i> (falsificación de peticiones del lado del servidor)
URL	<i>Uniform Resource Locator</i> (localizador uniforme de recursos)
VPS	<i>Virtual Private Server</i> (servidor privado virtual)
WCAG	<i>Web Content Accessibility Guidelines</i> (pautas de accesibilidad para el contenido web)

1. Introducción

1.1. Contexto y problema

La preparación de un ordenador nuevo, la reinstalación del sistema operativo o la configuración de un puesto de trabajo suelen exigir la descarga de numerosas aplicaciones. Aunque cada fabricante publica sus instaladores en páginas oficiales, el usuario debe localizar dichas páginas, identificar la versión adecuada para su sistema y repetir el proceso para cada programa. Esta tarea es mecánica, consume tiempo y aumenta la posibilidad de utilizar enlaces obsoletos o sitios no oficiales.

Batch Downloader se plantea como una plataforma web capaz de centralizar el descubrimiento y la descarga de software sin convertirse en un repositorio permanente de ejecutables. El catálogo mantendrá información descriptiva, metadatos y reglas para localizar los instaladores, pero los binarios se obtendrán desde las fuentes oficiales cuando el usuario solicite una descarga. El servidor reunirá temporalmente los archivos seleccionados, construirá un ZIP y eliminará los artefactos al finalizar.

El proyecto comparte con servicios como Ninite la idea de reducir las acciones necesarias para preparar un equipo. Ninite permite seleccionar varias aplicaciones y generar un instalador personalizado, descarga cada programa desde el sitio de su editor y verifica firmas o resúmenes antes de ejecutarlo [10]. Batch Downloader se diferencia por proponer bundles compartibles y editables de forma temporal, un catálogo administrable, compatibilidad planificada con distintos sistemas y arquitecturas, generación de ZIP en servidor y una capa de búsqueda semántica.

1.2. Motivación

La motivación principal es simplificar un proceso frecuente sin renunciar a la trazabilidad. El usuario debe conocer qué aplicaciones ha elegido, desde qué dominio se obtiene cada instalador y por qué un archivo no ha podido incluirse. Esta transparencia es especialmente importante porque la plataforma procesará ejecutables de terceros y dependerá de páginas externas que pueden cambiar sin previo aviso.

El sistema también aborda un problema de organización. Una selección habitual, como un entorno de desarrollo, un conjunto de utilidades multimedia o las aplicaciones básicas para un equipo nuevo, puede expresarse como un *bundle*. Los usuarios registrados podrán conservar sus bundles, publicarlos y marcar con una estrella los que les resulten útiles. Un bundle público podrá personalizarse para una descarga concreta sin modificar la versión almacenada; solamente su propietario podrá alterar de manera permanente la composición original.

Por último, las búsquedas exactas por nombre son insuficientes cuando el usuario no conoce una aplicación concreta. Una consulta como “programas para desarrollar en Java en Windows” expresa una necesidad, no un nombre. La búsqueda semántica permitirá recuperar aplicaciones relacionadas con esa intención y el modelo de lenguaje podrá explicar la recomendación. Esta capacidad se diseña como una ayuda subordinada a los datos y reglas de la plataforma, no como un componente con autoridad sobre las descargas.

1.3. Alcance de esta memoria

TODO: En el momento de redactar este documento el software aún no está implementado. La memoria recoge la fase de análisis y diseño: objetivos, conceptos, decisiones tecnológicas, arquitectura, requisitos, riesgos, planificación y criterios de validación. Los comportamientos se redactan, por tanto, como especificaciones previstas.

El alcance funcional incluye:

- consulta y filtrado de un catálogo de aplicaciones.
- selección individual o múltiple y generación de un ZIP en servidor.
- usuarios anónimos, registrados y administradores.
- bundles oficiales, públicos y privados.

- valoración binaria de bundles mediante estrellas.
- solicitud de nuevas aplicaciones y envío opcional de avisos por correo.
- resolución periódica de enlaces oficiales y revisión administrativa.
- búsqueda semántica, recomendaciones y asistencia para explicar errores.
- límites de uso, auditoría y medidas frente a descargas abusivas.

TODO: Quedan fuera de esta fase la implementación, las métricas reales de rendimiento y los manuales definitivos de programación y de usuario. Esos documentos se completarán cuando exista una versión ejecutable y puedan validarse los procedimientos contra el producto real.

1.4. Estructura del documento

- El capítulo 2 define los objetivos.
- El capítulo 3 presenta los conceptos necesarios para comprender la solución.
- El capítulo 4 justifica las tecnologías y técnicas seleccionadas.
- El capítulo 5 describe la arquitectura y los aspectos más relevantes del diseño.
- El capítulo 6 compara la propuesta con Ninite, Home Updater y UBULabs.
- El capítulo 7 resume las conclusiones alcanzadas durante el análisis y plantea líneas de trabajo futuras.
- La documentación técnica se entrega por separado. Sus anexos contienen el plan de proyecto, la especificación de requisitos, el diseño de datos y procedimientos, los esqueletos

2. Objetivos del proyecto

2.1. Objetivo general

El objetivo general es diseñar e implementar una plataforma web que permita localizar aplicaciones, seleccionar uno o varios instaladores y descargarlos agrupados desde sus fuentes oficiales, evitando el almacenamiento permanente de ejecutables y ofreciendo información clara sobre el origen y el estado de cada archivo.

2.2. Objetivos funcionales

1. Construir un catálogo administrable de software con nombre, descripción, etiquetas, página oficial, sistemas operativos, arquitecturas y estado de disponibilidad.
2. Permitir búsquedas parciales por nombre y filtros por etiquetas. El filtrado podrá exigir todas las etiquetas seleccionadas o un mínimo configurado por el usuario.
3. Permitir la selección de una o varias aplicaciones. Una selección múltiple se tratará como un bundle temporal para generar un ZIP.
4. Crear bundles oficiales, públicos y privados. Los usuarios anónimos podrán personalizar temporalmente los bundles públicos; los usuarios registrados podrán, además, guardar y publicar sus propios bundles.
5. Asignar un enlace estable a cada bundle público y ordenar el catálogo de bundles por popularidad, fecha, nombre o etiquetas.

6. Incorporar una estrella binaria por usuario y bundle. Cada usuario registrado podrá añadirla o retirarla una vez, y el número total servirá para ordenar los bundles.
7. Permitir a los usuarios registrados solicitar la incorporación de una aplicación, proporcionando al menos su nombre y su página oficial.
8. Proporcionar al administrador funciones para mantener aplicaciones, etiquetas, reglas de resolución, dominios permitidos y estados del catálogo.
9. Informar del progreso de cada descarga, de los archivos incluidos y de los errores que impidan completar una parte de la solicitud.

2.3. Objetivos de descarga y procesamiento

1. Resolver dinámicamente el enlace del instalador desde una fuente oficial mediante una estrategia específica o, como respaldo, mediante navegación automatizada con Playwright.
2. Validar protocolo, dominio final, redirecciones, tipo de contenido, extensión y tamaño antes de aceptar un archivo.
3. Descargar los instaladores en el servidor, generar un ZIP con un manifiesto y eliminar todos los archivos temporales al terminar, cancelar o expirar el trabajo.
4. Procesar las solicitudes de manera asíncrona con RabbitMQ para separar la interacción web de las operaciones de resolución, descarga y compresión.
5. Aplicar reintentos controlados y permitir un resultado parcial cuando una aplicación falle, manteniendo una explicación individual para cada incidencia.
6. Calcular, cuando las fuentes lo permitan, un tamaño estimado antes de iniciar el trabajo y aplicar límites por archivo, ZIP, usuario, dirección IP y número de operaciones concurrentes.

2.4. Objetivos de búsqueda semántica

1. Representar las aplicaciones mediante embeddings generados a partir de sus metadatos y almacenarlos en PostgreSQL mediante pgvector.
2. Interpretar consultas en lenguaje natural y recuperar candidatos semánticamente próximos antes de aplicar los filtros ordinarios del catálogo.
3. Utilizar un modelo de lenguaje para proponer bundles, explicar recomendaciones y resumir errores a partir de información recuperada del sistema.
4. Validar las salidas del modelo mediante esquemas estrictos y mantener desacoplado el proveedor para poder sustituir Groq por un modelo local o por otro servicio.
5. Impedir que el modelo resuelva o apruebe enlaces, cambie dominios autorizados, ejecute descargas o modifique reglas sin revisión administrativa.

2.5. Objetivos técnicos y de calidad

- Desarrollar un cliente ligero con Vite, HTML, CSS y TypeScript.
- Construir el backend con Java 26 y Spring Boot, separando las responsabilidades en servicios despleables y escalables.
- Documentar la API HTTP mediante OpenAPI, cuyo propósito es describir de forma independiente del lenguaje las capacidades de una API [7].
- Utilizar MySQL para la información transaccional y PostgreSQL con pgvector para la búsqueda vectorial.
- Contenerizar los componentes con Docker y preparar su despliegue compatible con Compose.
- Definir pruebas unitarias, de integración, de seguridad, de resolución de enlaces, de generación de ZIP y de extremo a extremo.
- Mantener trazabilidad mediante registros de administración, resolución, descargas y uso del asistente.

2.6. Objetivos de seguridad y sostenibilidad

El sistema deberá reducir los riesgos derivados de procesar URL externas y ejecutables. Para ello se definirán listas de dominios oficiales, bloqueo de redes privadas, límites de redirección, sanitización de nombres, protección frente a SSRF, control de acceso y cuotas de uso. La plataforma no ejecutará los instaladores descargados.

Desde el punto de vista de sostenibilidad, se buscará evitar copias permanentes, eliminar temporales, limitar trabajos innecesarios, reutilizar metadatos válidos y escalar los servicios según su carga. También se atenderá a la accesibilidad, la claridad de la información, la protección de datos y el acceso equitativo a la funcionalidad básica.

3. Conceptos teóricos

3.1. Catálogo centralizado y fuentes oficiales

Una plataforma de descarga puede almacenar los binarios o limitarse a mantener información para encontrarlos. Batch Downloader seguirá el segundo enfoque: conservará metadatos y reglas de resolución, mientras que los instaladores se obtendrán bajo demanda. Esta separación reduce la duplicación de ejecutables y evita que el catálogo se convierta en una fuente desactualizada respecto al editor original.

El concepto de *fente oficial* no se limita a la URL visible de una página. Una descarga puede atravesar redirecciones o terminar en una red de distribución de contenidos. Por ello, cada aplicación tendrá una lista de dominios permitidos y la URL final se comprobará antes de aceptar el archivo. La interfaz mostrará tanto la página oficial como el estado y la fecha de la última comprobación disponible.

3.2. Bundles y personalización temporal

Un bundle es una colección identificable de aplicaciones. Puede representar una necesidad concreta, por ejemplo “herramientas para desarrollo web”, o ser simplemente una selección personal. El modelo distingue:

Bundle oficial: creado y mantenido por un administrador.

Bundle público: creado por un usuario registrado y visible mediante un enlace compartible.

Bundle privado: accesible únicamente para su propietario.

Bundle temporal: selección no persistente utilizada para una descarga concreta.

La personalización temporal evita confundir la composición publicada con las preferencias de quien la descarga. Al abrir un bundle público, cualquier usuario podrá añadir o retirar aplicaciones antes de generar su ZIP. Esos cambios vivirán solamente en la solicitud actual. La edición persistente seguirá reservada al propietario o, en el caso de bundles oficiales, a un administrador.

3.3. Resolución de enlaces y scraping

Las URL de descarga cambian por versiones, regiones, sistemas operativos o decisiones del proveedor. Un *resolver* es el componente que transforma la información de una aplicación y el contexto solicitado en una URL descargable validada.

La estrategia prevista será progresiva:

1. utilizar una URL directa configurada y todavía válida;
2. utilizar un resolver específico para el proveedor;
3. navegar por la página oficial mediante Playwright, inspeccionando enlaces, botones, redirecciones y tráfico de red;
4. marcar la fuente para revisión manual si no se obtiene un resultado seguro.

Playwright permite controlar navegadores modernos desde Java y ejecutarlos en modo sin interfaz [6]. Su uso será apropiado para páginas que generan los enlaces mediante JavaScript o requieren interacción. La navegación automatizada no elimina las comprobaciones posteriores: el resultado deberá seguir utilizando HTTPS, pertenecer a un dominio permitido y cumplir los límites establecidos.

3.4. Procesamiento asíncrono

Resolver varias páginas, descargar archivos grandes y comprimirlos puede superar el tiempo razonable de una petición HTTP. El procesamiento asíncrono separa la creación de la solicitud de su ejecución.

El backend validará la selección y publicará un mensaje en RabbitMQ. Un worker consumirá ese mensaje y actualizará el estado del trabajo a medida que avance. Spring AMQP proporciona abstracciones para enviar y recibir mensajes y para implementar consumidores dirigidos por mensajes [1]. El usuario consultará el progreso mediante la API y descargará el ZIP cuando el estado sea *ready*.

Los estados previstos para un trabajo son:

Estado	Significado
<i>queued</i>	Espera de un worker disponible.
<i>resolving</i>	Resolución y validación de fuentes.
<i>downloading</i>	Descarga temporal de instaladores.
<i>compressing</i>	Construcción del ZIP y del manifiesto.
<i>ready</i>	Archivo preparado para su entrega.
<i>failed</i>	No se ha podido producir un resultado válido.
<i>cancelled</i>	Trabajo cancelado por el usuario o el sistema.
<i>expired</i>	Resultado eliminado al superar su tiempo de vida.

3.5. Generación temporal del ZIP

Cada trabajo utilizará un espacio aislado identificado por su código. Los instaladores no se incorporarán a un almacenamiento permanente del catálogo. MinIO podrá emplearse como almacenamiento de objetos temporal para compartir artefactos entre workers, aplicar tiempos de expiración y evitar que el servidor web mantenga archivos grandes en su sistema local.

El ZIP incluirá una carpeta con los instaladores disponibles, un fichero *manifest.json* y un documento de lectura. El manifiesto registrará la aplicación, la fuente oficial, el sistema operativo, la arquitectura, el nombre del archivo y el resultado de la validación. Si alguna descarga falla, se añadirá una explicación y el ZIP podrá completarse con los archivos restantes.

3.6. Embeddings y búsqueda vectorial

Un embedding es una representación numérica de un elemento en un espacio vectorial. Textos con significado relacionado tienden a ocupar posiciones próximas según una medida de distancia. Para cada aplicación se generará un vector a partir de su nombre, descripción, categorías, sistemas soportados, arquitectura y otros metadatos relevantes.

PostgreSQL con pgvector permite almacenar estos vectores junto con los datos de referencia y realizar búsquedas de vecinos cercanos mediante distancia euclídea, producto interno o similitud coseno [5]. Ante una consulta, el sistema generará su embedding y recuperará los candidatos más próximos. Después aplicará reglas deterministas: compatibilidad de sistema y arquitectura, estado activo, restricciones de seguridad y disponibilidad.

La búsqueda semántica complementará, pero no sustituirá, la búsqueda por nombre. Las consultas exactas o parciales seguirán resolviéndose de forma convencional porque son más predecibles y económicas. La búsqueda vectorial se utilizará cuando la consulta exprese una intención, cuando no existan coincidencias suficientes o cuando el usuario active explícitamente el modo semántico.

3.7. Modelo de lenguaje y generación aumentada con recuperación

El modelo de lenguaje recibirá únicamente contexto recuperado desde el catálogo, los estados de los trabajos o registros internos autorizados. Este patrón, conocido como generación aumentada con recuperación, reduce la necesidad de confiar en conocimiento no verificable del modelo.

Durante el desarrollo se prevé utilizar Groq como proveedor de inferencia, cuya API ofrece compatibilidad con clientes del ecosistema OpenAI [4]. La integración se encapsulará detrás de una interfaz propia para poder cambiar de proveedor o ejecutar un modelo local sin modificar el resto de la aplicación.

Las respuestas se validarán contra esquemas estructurados antes de mostrarse o transformarse en acciones. Las responsabilidades del modelo quedarán limitadas a:

- explicar por qué unas aplicaciones se ajustan a una consulta;
- sugerir una selección inicial para un bundle;
- resumir errores técnicos en un lenguaje comprensible;
- ayudar al administrador a clasificar incidencias o detectar entradas similares.

El modelo no podrá aprobar dominios, resolver una URL por sí solo, ejecutar descargas, cambiar reglas de scraping, generar un ZIP ni modificar el catálogo sin una operación autorizada y validada por los servicios correspondientes.

3.8. Seguridad en la descarga de recursos externos

El backend recibirá y seguirá URL externas, lo que introduce riesgo de falsificación de peticiones del lado del servidor o SSRF. Un atacante podría intentar acceder a servicios internos, metadatos de infraestructura o redes privadas. La mitigación prevista incluye:

- admitir exclusivamente HTTPS;
- resolver DNS y rechazar direcciones locales, privadas, de enlace local o reservadas;
- repetir la validación después de cada redirección;
- aceptar solo dominios incluidos en la configuración de la aplicación;
- limitar redirecciones, tiempo, tamaño y concurrencia;
- no ejecutar ningún archivo y sanitizar su nombre antes de incorporarlo al ZIP.

También se aplicarán controles de autenticación, autorización, limitación de frecuencia, cuotas y auditoría. Estas medidas evitan que la plataforma se utilice como proxy de descargas masivas y proporcionan evidencias para investigar incidencias.

4. Técnicas y herramientas

4.1. Criterios de selección

TODO: Las tecnologías se han seleccionado buscando tipado estático, separación de responsabilidades, facilidad de despliegue, observabilidad y posibilidad de escalar los procesos costosos de forma independiente. Dado que esta memoria describe una arquitectura planificada, las elecciones deberán confirmarse con prototipos durante la implementación.

4.2. Frontend

Vite

Vite se utilizará como herramienta de desarrollo y construcción del cliente. Proporciona un servidor de desarrollo basado en módulos ES y un proceso de construcción optimizado para producción [12]. La propuesta evita introducir un framework de interfaz completo desde el inicio y mantiene el frontend próximo a las tecnologías web estándar.

HTML, CSS y TypeScript

HTML definirá una estructura semántica y accesible; CSS permitirá una interfaz compacta capaz de mostrar muchas aplicaciones; y TypeScript aportará tipos a modelos, respuestas de la API, estados de trabajos y formularios. El tipado compartido reducirá errores al evolucionar el contrato entre el frontend y el backend.

4.3. Backend

Java 26

TODO: Java será el lenguaje principal del servidor por su sistema de tipos, madurez, bibliotecas y experiencia previa del autor. Se plantea utilizar Java 26, versión de la plataforma Java SE publicada en marzo de 2026 [8]. Antes de comenzar la implementación se verificará la compatibilidad de Spring Boot y de las dependencias elegidas; si fuera necesario se adoptará una versión LTS compatible sin cambiar el diseño.

Spring Boot

Spring Boot facilita la creación de aplicaciones autónomas con configuración inicial reducida y características de producción como métricas y comprobaciones de salud [2]. Se prevén los siguientes módulos:

Spring Web: API REST, validación del protocolo HTTP y entrega de resultados.

Spring Security: autenticación, sesiones o tokens, roles y protección de rutas.

Spring Data JPA: persistencia transaccional y repositorios.

Spring Validation: validación declarativa de entradas y configuraciones.

Spring Actuator: salud, métricas y observabilidad de servicios.

Spring AMQP: publicación y consumo de trabajos mediante RabbitMQ.

OpenAPI

OpenAPI se empleará para describir el contrato de la API HTTP. La especificación permite que personas y herramientas comprendan las operaciones de un servicio sin inspeccionar su código [7]. La descripción servirá para generar documentación, validar compatibilidad y facilitar clientes de prueba. No se utilizará para realizar scraping.

4.4. Resolución de fuentes

TODO: Playwright para Java

Playwright controlará un navegador en modo *headless* para páginas que requieran JavaScript, interacción o inspección de red. Su API para Java se distribuye mediante módulos Maven y permite manejar Chromium, Firefox y WebKit [6]. Para cada proveedor se intentará limitar la automatización a pasos conocidos y selectores revisables.

La navegación generalista se mantendrá como mecanismo de respaldo. Los proveedores con procesos complejos podrán disponer de resolvers específicos, de forma que los cambios en una página no afecten al resto del catálogo.

TODO: Watcher en Python

Un watcher que cada cierto tiempo se ejecuta sobre las URLs para poder obtener información de las fuentes y comprobar si hay cambios.

4.5. Persistencia

MySQL

MySQL almacenará los datos transaccionales: usuarios, aplicaciones, fuentes, bundles, estrellas, solicitudes, trabajos, estados y registros de auditoría. La elección mantiene un modelo relacional apropiado para restricciones de integridad, relaciones entre entidades y operaciones administrativas.

PostgreSQL y pgvector

La búsqueda semántica se aislará en PostgreSQL con pgvector. Esta extensión añade tipos y operadores para búsqueda exacta y aproximada de vectores [5]. Separar este almacenamiento permite evolucionar la parte semántica sin condicionar el modelo transaccional principal.

La duplicación de información se limitará a los identificadores y metadatos necesarios para generar embeddings. MySQL seguirá siendo la fuente de verdad del catálogo y cualquier vector incluirá la versión o fecha del contenido del que procede.

4.6. Mensajería y almacenamiento temporal

RabbitMQ

RabbitMQ desacoplará la API de los workers encargados de resolver, descargar y comprimir. Spring AMQP ofrece una abstracción de alto nivel para soluciones basadas en AMQP y consumidores asíncronos [1]. Se definirán colas o claves de enrutamiento para generación de ZIP, comprobación periódica de fuentes y notificaciones.

Los mensajes contendrán identificadores, no datos sensibles ni archivos. El estado persistente del trabajo permitirá reintentos controlados y evitará depender exclusivamente de la cola.

MinIO

MinIO se reservará para objetos temporales como ZIP preparados, manifiestos o archivos intermedios que deban compartirse entre instancias. Las políticas de expiración eliminarán resultados no descargados. No se utilizará como repositorio permanente de instaladores ni como base de conocimiento para el modelo.

4.7. Búsqueda semántica y LLM

Embeddings

Un servicio de embeddings convertirá las consultas y descripciones del catálogo en vectores compatibles. La interfaz del servicio incluirá el identificador del modelo y la dimensión para poder regenerar los datos cuando cambie la representación.

Groq y desacoplamiento del proveedor

TODO: Groq se propone como proveedor durante el desarrollo por su rapidez de inferencia y su API compatible con clientes OpenAI [4]. El backend no expondrá directamente el SDK al resto de módulos: utilizará puertos propios para generación de embeddings y respuestas. Esto permitirá sustituir el servicio por un modelo local, otra nube o una implementación de prueba. TODO: Hacerlo compatible también con openrouter, aunque utilizando el patrón strategy para poder cambiar de proveedor sin modificar el resto de la aplicación.

4.8. Contenedores y despliegue

Docker

Cada componente se empaquetará en una imagen Docker. Los contenedores aíslan aplicaciones y dependencias y permiten ejecutar la misma unidad en diferentes entornos [3]. Para desarrollo se preparará una composición con frontend, API, workers, bases de datos, RabbitMQ y MinIO.

4.9. Herramientas de desarrollo y documentación

Git se utilizará para control de versiones; Maven gestionará dependencias y construcción del backend; npm gestionará el frontend; y $\text{\LaTeX}$ se utilizará para mantener la memoria y la documentación técnica. Mermaid será la fuente editable de los diagramas de secuencia, que se exportarán a un formato vectorial antes de incorporarlos al documento.

4.10. Resumen de correspondencia

Necesidad	Tecnología propuesta
Cliente web	Vite, HTML, CSS y TypeScript
API y lógica de negocio	Java 26 y Spring Boot
Contrato HTTP	OpenAPI
Datos transaccionales	MySQL
Búsqueda vectorial	PostgreSQL y pgvector
Trabajos asíncronos	RabbitMQ y Spring AMQP
Scraping dinámico	Playwright para Java
Objetos temporales	MinIO con expiración
Asistencia semántica	Embeddings y Groq mediante una interfaz desacoplada
Portabilidad y escalado	Docker
Documentación	$\text{\LaTeX}$ y Mermaid

Tabla 4.1: Tecnologías propuestas por responsabilidad.

5. Aspectos relevantes del desarrollo del proyecto

5.1. Enfoque de desarrollo

El proyecto se abordará de forma incremental. Antes de implementar la búsqueda semántica o el despliegue distribuido será necesario validar el flujo esencial: mantener un catálogo, resolver una fuente oficial, descargar un archivo de forma segura, producir un ZIP y eliminar los temporales.

Las iteraciones previstas son:

1. catálogo, roles y búsqueda convencional;
2. selección de aplicaciones y generación asíncrona de ZIP;
3. bundles, estrellas y administración;
4. resolvers específicos, comprobaciones periódicas y observabilidad;
5. embeddings, búsqueda semántica y asistencia con LLM;
6. contenerización, pruebas de carga y despliegue escalable.

Esta secuencia reduce el riesgo de integrar inteligencia artificial sobre un núcleo todavía inestable. Cada iteración deberá producir un flujo demostrable y pruebas automatizadas.

5.2. Actores y permisos

Acción	Anónimo	Registrado	Administrador
Consultar catálogo	Sí	Sí	Sí
Generar ZIP	Sí, con cuotas	Sí, con cuotas propias	Sí
Usar búsqueda semántica	Sí, limitada	Sí	Sí
Personalizar bundle público	Temporalmente	Temporalmente	Temporalmente
Guardar bundle	No	Sí	Sí
Publicar bundle	No	Sí	Sí
Asignar estrella	No	Sí	Sí
Solicitar software	No	Sí	Sí
Mantener catálogo y resolvers	No	No	Sí
Revisar registros	No	Solo los propios	Sí

Tabla 5.1: Permisos generales por actor.

La personalización temporal no otorga permisos de edición. Al descargar un bundle se construirá una copia de su composición para la solicitud actual. Únicamente el propietario podrá guardar cambios permanentes y los bundles oficiales quedarán bajo control administrativo.

5.3. Arquitectura planificada

La solución seguirá una arquitectura de servicios con responsabilidades delimitadas:

Frontend: presenta catálogo, selección, bundles, progreso y administración.

API: valida entradas, aplica permisos, mantiene entidades y publica trabajos.

Servicio de catálogo: gestiona aplicaciones, fuentes, etiquetas y estados.

Servicio de bundles: gestiona composición, visibilidad, enlaces y estrellas.

Servicio de resolución: selecciona el resolver, controla Playwright y valida URL.

Workers de descarga: consumen trabajos, descargan archivos y notifican progreso.

Generador de ZIP: crea el paquete, el manifiesto y los informes de error.

Servicio semántico: genera embeddings, consulta pgvector y coordina el LLM.

Servicio de notificaciones: envía avisos cuando exista consentimiento y configuración.

Los servicios podrán desplegarse inicialmente dentro de una aplicación modular. La separación lógica se mantendrá para permitir que las tareas intensivas se extraigan a procesos independientes cuando la carga lo justifique.

5.4. Flujo general de descarga

La figura 5.1 resume el recorrido desde la selección del usuario hasta la eliminación de los temporales.

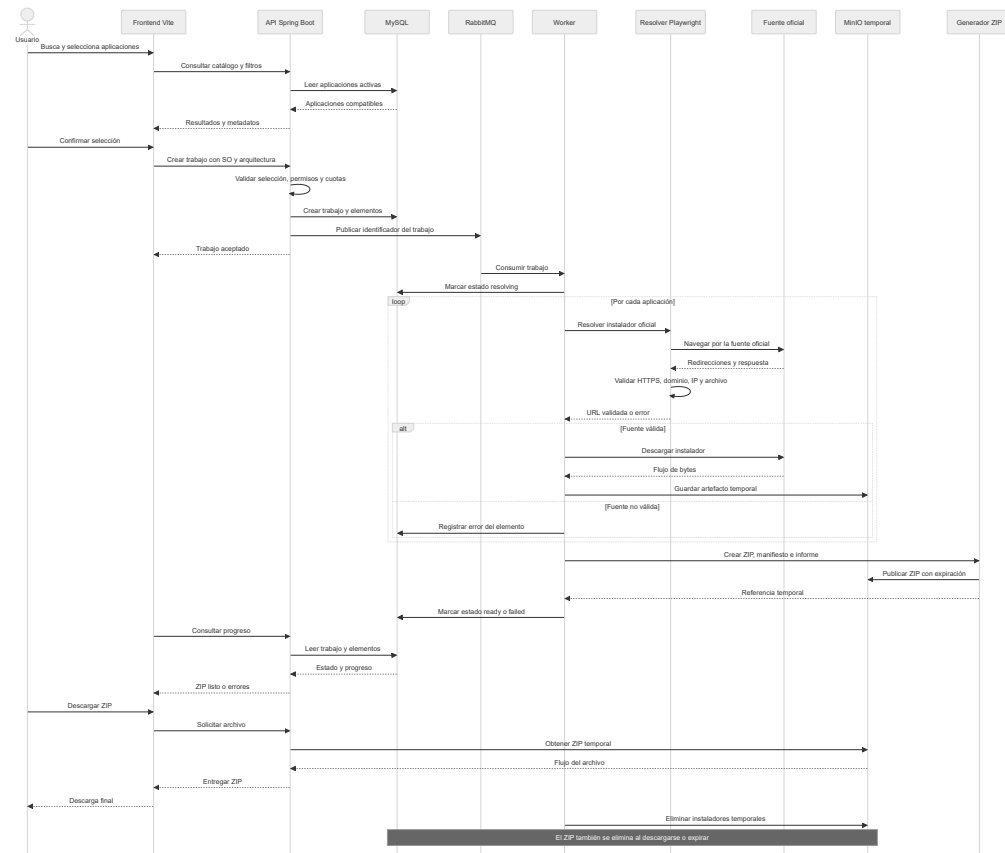


Figura 5.1: Secuencia general de generación y descarga de un ZIP.

El backend no devolverá el ZIP en la misma petición que crea el trabajo. Tras validar la selección y las cuotas, persistirá el estado inicial y publicará un mensaje en RabbitMQ. La respuesta inmediata contendrá un identificador que el frontend utilizará para consultar el progreso.

El worker procesará cada aplicación de forma aislada. Una incidencia no debe ocultar el estado de las demás. Cuando haya al menos un archivo válido, el trabajo podrá producir un ZIP parcial acompañado de errores legibles. Si no se incluye ningún archivo, el estado será *failed*.

5.5. Resolución de instaladores

Cada fuente combinará datos declarativos y una estrategia:

- sistema operativo, arquitectura, región y país;
- página oficial de partida;
- dominios permitidos;
- tipo de resolver y configuración;
- extensiones y tipos de contenido esperados;
- tiempo máximo, reintentos y número de redirecciones;
- fecha y resultado de la última comprobación.

El *DefaultResolver* intentará una resolución genérica con Playwright. Abrirá la página oficial, buscará elementos configurados, observará respuestas de red y detectará descargas. Los proveedores con flujos estables pero particulares dispondrán de resolvers dedicados.

El resultado del navegador no será suficiente. Un validador independiente repetirá las comprobaciones de dominio y red, realizará una petición controlada y verificará que la respuesta se corresponde con un instalador esperado. Esta separación evita que un error en el scraping se convierta directamente en una descarga.

5.6. Generación y contenido del ZIP

El trabajo dispondrá de un espacio temporal aislado. Los nombres de archivo se normalizarán, se eliminarán componentes de ruta y se resolverán duplicados. El contenido previsto será:

```
apps/  
  instalador-aplicacion.exe  
  instalador-aplicacion2.exe  
errors/  
  aplicacion-no-incluida.txt  
manifest.json  
README.txt
```

El manifiesto indicará la fecha de creación, la política de fuentes oficiales, la naturaleza temporal del procesamiento y el resultado de cada elemento. La URL resuelta podrá omitirse o protegerse en la entrega si contiene credenciales temporales; la página oficial y el dominio final sí deberán quedar identificados.

El ZIP tendrá una fecha de expiración. Después de su descarga o al superar ese plazo, el objeto se eliminará. Una tarea periódica limpiará trabajos abandonados, archivos incompletos y objetos cuyo estado no coincida con la base de datos.

5.7. Bundles y estrellas

La figura 5.2 muestra la creación, publicación, personalización y descarga de un bundle.

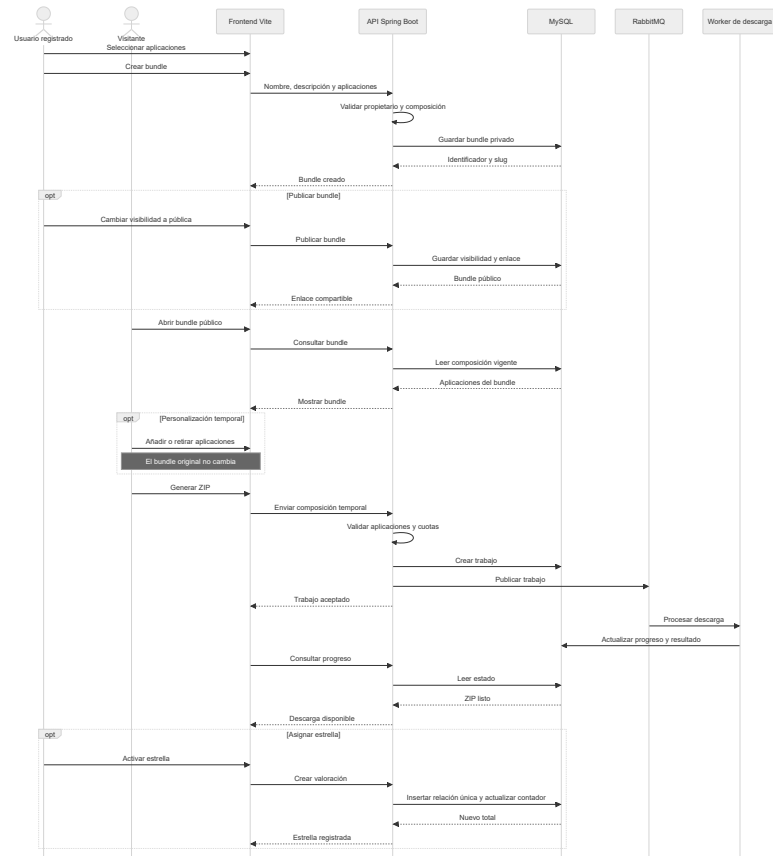


Figura 5.2: Secuencia de creación y descarga de un bundle.

La visibilidad se modelará explícitamente como privada o pública. La marca de bundle oficial será independiente de la visibilidad y requerirá rol de administrador. Si un bundle público pasa a privado, su enlace dejará de ser accesible y se conservará un registro de la operación. Las estrellas dejarán de participar en los rankings mientras no sea público.

La valoración será binaria: un usuario puede asignar o retirar una estrella. La restricción única entre usuario y bundle impedirá duplicados. El contador agregado podrá almacenarse para ordenar resultados, pero deberá ser recalculable desde las valoraciones individuales.

5.8. Búsqueda semántica y asistencia

La figura 5.3 refleja el flujo de una consulta expresada en lenguaje natural.

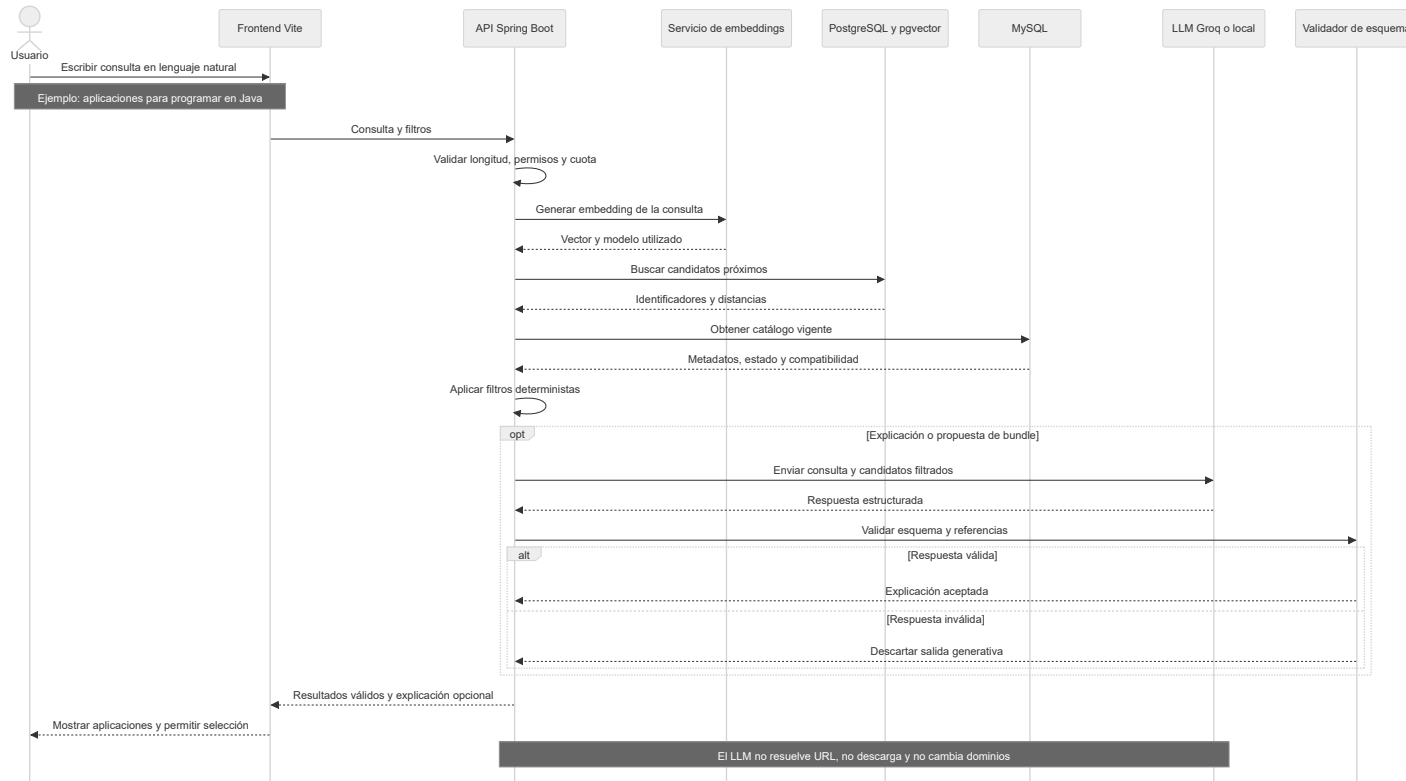


Figura 5.3: Secuencia de búsqueda semántica y explicación mediante LLM.

La API generará el embedding de la consulta y recuperará un conjunto limitado de candidatos en pgvector. A continuación obtendrá la versión vigente de cada aplicación desde MySQL y descartará elementos incompatibles, inactivos o no disponibles.

El LLM recibirá la consulta, los candidatos ya filtrados y un esquema de salida. Podrá ordenar o explicar resultados, pero la lista final no podrá contener aplicaciones ausentes del contexto recuperado. Si la validación falla o el proveedor no responde, el usuario seguirá recibiendo los resultados vectoriales sin explicación generativa.

El mismo patrón se aplicará a la creación asistida de bundles. La propuesta será un borrador editable; la selección se validará de nuevo antes de guardarse o descargarse.

5.9. Seguridad

Prevención de SSRF

Las URL externas se considerarán datos no confiables. Antes de cualquier conexión se validarán esquema, host y puerto; se resolverán todas las direcciones; y se rechazarán redes privadas, loopback, enlace local, direcciones reservadas y puntos de metadatos de proveedores cloud. La validación se repetirá tras cada redirección para reducir ataques de cambio de DNS.

Allowlist de dominios

Cada fuente tendrá una lista explícita de dominios oficiales. Se compararán nombres normalizados y límites de subdominio, evitando aceptar coincidencias parciales como `dominio-oficial.ejemplo-malicioso.com`. Las modificaciones de la lista requerirán privilegios administrativos y quedarán auditadas.

Límites y protección frente a abuso

Se configurarán máximos de aplicaciones por ZIP, tamaño por archivo, tamaño total, tiempo del trabajo, redirecciones, reintentos, descargas paralelas y trabajos activos. Los límites se aplicarán por dirección IP, sesión anónima, usuario y, si se habilitan integraciones, clave de API.

Los usuarios anónimos conservarán la funcionalidad básica con cuotas inferiores. Podrá activarse un CAPTCHA cuando se detecten patrones abusivos, sin convertirlo en requisito permanente para el uso normal.

Archivos y ZIP

La plataforma no ejecutará los instaladores. Los nombres se sanitizarán para impedir rutas absolutas, secuencias ../, nombres reservados o colisiones. Se validará el tamaño real y se interrumpirá la descarga si supera el límite, aunque la fuente hubiera anunciado un valor menor.

Autenticación y autorización

Spring Security protegerá las operaciones por rol y propietario. Los permisos no dependerán de botones ocultos en el frontend. Cada modificación de catálogo, resolver, dominio o bundle oficial será verificada en el backend y registrada con usuario, fecha, objetivo y datos relevantes.

5.10. Gestión de errores

Los errores se clasificarán por fase:

Resolución: no existe enlace, cambió la página, se requiere interacción no soportada o el dominio final no es válido.

Validación: protocolo, tamaño, extensión, tipo o redirección no permitidos.

Descarga: timeout, conexión interrumpida, límite externo o archivo incompleto.

Compresión: falta de espacio, cancelación o error al construir el ZIP.

Asistencia: proveedor no disponible, esquema inválido o contexto insuficiente.

El usuario recibirá una explicación comprensible y un identificador de incidencia, mientras que el registro técnico conservará detalles para el administrador sin exponer secretos ni trazas internas.

5.11. Observabilidad

Spring Actuator y las métricas de los componentes permitirán observar:

- trabajos creados, completados, parciales, fallidos y expirados;
- duración de resolución, descarga y compresión;
- longitud de las colas y número de consumidores;
- aplicaciones con más fallos y dominios que cambian;
- espacio temporal utilizado y objetos pendientes de limpieza;
- consultas semánticas, fallos del proveedor y validaciones rechazadas.

Estas métricas no se presentan como resultados del proyecto; constituyen el plan de instrumentación que deberá implementarse y evaluarse.

5.12. Estrategia de pruebas

La calidad se verificará en varios niveles:

- pruebas unitarias de URL, dominios, IP privadas, nombres, permisos, estrellas y estados;
- pruebas de integración con MySQL, PostgreSQL, RabbitMQ y MinIO;
- páginas controladas para probar resolvers, redirecciones y descargas;
- pruebas de ZIP correcto, parcial, cancelado y expirado;
- pruebas de seguridad para SSRF, cuotas, autorización y datos maliciosos;
- pruebas de extremo a extremo del catálogo, selección, progreso, bundles y administración;
- pruebas de carga que midan concurrencia, recursos y comportamiento del escalado.

La aceptación de la primera versión requerirá demostrar que los instaladores proceden de dominios autorizados, que ningún temporal permanece después del plazo establecido y que un fallo individual no provoca estados incoherentes.

6. Trabajos relacionados

6.1. Ninite

Ninite es la referencia más cercana a la idea central del proyecto. Su página permite marcar aplicaciones y obtener un instalador personalizado que automatiza el proceso. El servicio declara que descarga las aplicaciones desde el sitio de cada editor, selecciona versiones adecuadas, omite programas ya actualizados y verifica firmas digitales o resúmenes antes de ejecutar [10].

Su propuesta destaca por la sencillez: el usuario elige programas, descarga un ejecutable y lo inicia. También dispone de una oferta profesional orientada a la administración y actualización de equipos.

6.2. Patch My PC Home Updater

Patch My PC Home Updater es una aplicación gratuita para Windows orientada a instalar, actualizar y desinstalar software de uso habitual. La herramienta mantiene un catálogo de más de 500 aplicaciones y permite ejecutar instalaciones y actualizaciones silenciosas, programar comprobaciones automáticas y excluir aquellos programas que el usuario no desee actualizar. También ofrece una versión portable y una función de desinstalación múltiple [9].

Su propuesta está especialmente centrada en el mantenimiento continuo del equipo. Las actualizaciones se prueban para comprobar su detección e instalación y, según el proveedor, los archivos se verifican mediante Virus-Total. Además, la aplicación evita incluir software adicional no solicitado y puede impedir reinicios durante el proceso de actualización [9].

La principal diferencia respecto a Batch Downloader es la necesidad de ejecutar un agente local. Patch My PC administra directamente el software instalado en el equipo y automatiza operaciones que pueden requerir el cierre de aplicaciones o permisos de instalación. Batch Downloader, por el contrario, se limita a localizar y agrupar instaladores oficiales en un archivo ZIP, sin instalar, actualizar ni desinstalar programas.

6.3. UBULabs

UBULabs es un servicio de la Universidad de Burgos que proporciona a la comunidad universitaria acceso a las aplicaciones necesarias para prácticas docentes y laboratorios. El usuario accede al portal mediante sus credenciales institucionales y el catálogo disponible depende de su cuenta, del sistema operativo y de la red desde la que se conecta [11]. Algunas aplicaciones solo están disponibles desde la red de la Universidad debido a las condiciones de sus licencias.

La plataforma utiliza dos mecanismos de entrega. Mediante el *streaming* de aplicaciones se despliegan programas preconfigurados desde los servidores de la Universidad para que se ejecuten en el equipo del usuario. Cuando una licencia está restringida a la red universitaria o el acceso se realiza desde el exterior, se recurre a la virtualización: la aplicación se ejecuta en un servidor y el usuario interactúa con ella mediante un cliente remoto [11].

UBULabs comparte con Batch Downloader la idea de ofrecer un punto de acceso centralizado a un catálogo de software, pero responde a un contexto institucional. Su catálogo y sus permisos están condicionados por acuerdos de licencia, asignaturas y pertenencia a la Universidad. Batch Downloader plantea, en cambio, un catálogo público de instaladores obtenidos desde los sitios oficiales, bundles compartibles y descargas temporales, sin proporcionar licencias ni ejecutar las aplicaciones en infraestructura propia.

6.4. Comparación funcional

Característica	Ninite / Patch My PC	UBULabs	Batch Downloader
Finalidad principal	Instalación y mantenimiento automatizados	Acceso académico a software con licencia	Descarga conjunta de instaladores oficiales
Selección múltiple	Sí	No	Sí
Forma de entrega	Instalador o agente local	<i>Streaming</i> , descarga o escritorio virtual	ZIP temporal con instaladores y manifiesto
Ejecución automática	Sí	Sí, local o remota según la aplicación	No; el usuario decide qué ejecutar
Control de acceso	Servicio general, con oferta profesional	Credenciales, permisos y condiciones de red	Catálogo público y funciones asociadas a cuentas
Persistencia de binarios	Gestionada por cada servicio	Aplicaciones alojadas o desplegadas por la Universidad	Prohibida por diseño para instaladores
Colecciones compartibles	Selección de aplicaciones	No es su objetivo	Bundles oficiales, públicos y privados
Participación comunitaria	Sugerencia de aplicaciones	Solicitud institucional de aplicaciones	Publicación, clonación y estrellas
Sistemas operativos	Principalmente Windows	Depende de la aplicación y del método de acceso	TODO: Windows, Linux y macOS como alcance planificado
Búsqueda semántica	No	No	Embeddings y asistencia con LLM

Tabla 6.1: Comparación conceptual de las soluciones relacionadas con Batch Downloader.

6.5. Diferencias de enfoque

Ninite y Patch My PC buscan que la instalación o actualización sea automática y silenciosa. Batch Downloader se centrará en reunir archivos y conservar trazabilidad, sin ejecutar software en el equipo del usuario. Esta decisión reduce el alcance y evita desarrollar un agente local con privilegios de instalación, aunque traslada al usuario la ejecución posterior.

UBULabs resuelve un problema diferente: garantizar que una comunidad concreta pueda utilizar software sujeto a licencias institucionales. Su combinación de despliegue y virtualización permite ejecutar aplicaciones incluso cuando no pueden instalarse libremente en el equipo del usuario. Batch Downloader no cubrirá este escenario, ya que solo incluirá software cuyos instaladores puedan obtenerse legítimamente desde fuentes oficiales y no gestionará licencias de uso.

Los bundles introducen una capa social y organizativa. Un enlace podrá representar una selección recomendada, y cada receptor podrá adaptarla sin alterar el original. Las estrellas permitirán destacar colecciones útiles, mientras que los administradores mantendrán bundles oficiales para casos comunes.

La búsqueda semántica también amplía el modelo de catálogo. En lugar de exigir que el usuario conozca nombres concretos, permitirá expresar objetivos. El resultado seguirá dependiendo de aplicaciones registradas y validadas, por lo que no se convertirá en una búsqueda abierta de ejecutables en Internet.

6.6. Lecciones aplicadas al diseño

La simplicidad de Ninite y Patch My PC demuestra la importancia de reducir pasos y explicar claramente qué ocurrirá. Batch Downloader deberá evitar que la complejidad interna de colas, workers y resolvers se traslade a la interfaz. El flujo principal seguirá siendo seleccionar, revisar y generar.

También resulta relevante la confianza. Ninite destaca el origen oficial y la verificación de los archivos [10], mientras que Patch My PC añade pruebas previas de las actualizaciones [9]. En Batch Downloader estas garantías deberán materializarse mediante dominios permitidos, validación de redirecciones, manifiesto, fechas de comprobación y mensajes explícitos.

UBULabs muestra, además, que la disponibilidad de una aplicación puede depender del usuario, la plataforma, la ubicación de red y la licencia. Aunque Batch Downloader no gestionará licencias institucionales, su catálogo deberá representar con claridad los sistemas operativos compatibles y cualquier restricción relevante antes de iniciar una descarga.

Por último, la comparación ayuda a delimitar el proyecto. Batch Downloader no pretende sustituir un gestor de paquetes del sistema operativo ni automatizar instalaciones con privilegios. Su aportación se sitúa en la preparación de una descarga conjunta, la gestión de bundles y la recuperación semántica dentro de un catálogo controlado.

7. Conclusiones y líneas de trabajo futuras

7.1. Conclusiones de la fase de análisis y diseño

La fase documentada permite concretar Batch Downloader como algo más que una página con enlaces. La generación conjunta de instaladores exige resolver fuentes cambiantes, validar destinos, procesar archivos grandes de forma asíncrona, comunicar progreso y eliminar temporales. Estos requisitos justifican la separación entre API, workers, resolvers y almacenamiento temporal.

La decisión de no conservar instaladores permanentemente se mantiene como principio central. El catálogo almacenará conocimiento sobre las aplicaciones y sus fuentes, pero cada archivo se obtendrá en el contexto de un trabajo. Esta política reduce duplicación y favorece la actualización, aunque obliga a asumir que las páginas externas pueden cambiar y que será necesario mantener resolvers y registros.

Los bundles aportan reutilización sin perder control sobre la versión publicada. La personalización temporal permite adaptar una colección a una descarga concreta y la restricción de edición permanente protege la autoría del bundle. La estrella binaria ofrece un mecanismo sencillo de popularidad sin convertir el proyecto en una plataforma de reseñas.

La búsqueda semántica resulta útil para consultas por intención, pero solo si permanece subordinada al catálogo y a las reglas deterministas. El modelo de lenguaje se ha delimitado como herramienta explicativa y de

propuesta. No tendrá capacidad para aprobar enlaces, descargar archivos ni modificar configuraciones críticas.

La arquitectura propuesta es amplia y deberá implementarse por etapas. La prioridad será validar un núcleo seguro y observable antes de introducir el LLM. Esta conclusión evita que componentes llamativos oculten los riesgos esenciales del tratamiento de URL y ejecutables externos.

7.2. Limitaciones actuales

TODO: El documento no incluye mediciones reales, porque todavía no existe una implementación. Tampoco se ha comprobado la estabilidad de los resolvers frente a proveedores concretos, el consumo de disco durante descargas concurrentes ni la calidad de los embeddings sobre un catálogo real.

TODO: La compatibilidad con Windows, Linux y macOS dependerá de la disponibilidad y condiciones de las fuentes oficiales. Algunos proveedores pueden exigir autenticación, aceptación manual de condiciones, elección regional o uso de tiendas propietarias. Esos casos deberán marcarse como no automatizables o requerir una estrategia específica.

TODO: La elección de Java 26 y de versiones concretas de las herramientas deberá revisarse al iniciar el desarrollo. La arquitectura no depende de una versión puntual, pero las combinaciones de Spring Boot, controladores y bibliotecas sí requieren una matriz compatible.

7.3. Líneas de trabajo futuras

Primera versión funcional

La primera línea de trabajo será construir un producto mínimo con catálogo, búsqueda por nombre, autenticación, URL directas, Playwright como respaldo, RabbitMQ, generación de ZIP, bundles, estrellas y registros básicos. Las pruebas de SSRF y limpieza temporal serán criterios de aceptación, no mejoras opcionales.

Resolvers y salud del catálogo

Se podrán desarrollar resolvers específicos para proveedores frecuentes, comprobaciones programadas y un panel que priorice aplicaciones rotas.

El historial permitirá detectar cambios repetidos y decidir qué entradas necesitan mantenimiento manual.

Rendimiento y streaming

Tras medir el uso de disco y memoria se estudiará añadir archivos al ZIP mediante streaming. Esta técnica reduciría almacenamiento temporal, pero requiere resolver cuidadosamente los fallos parciales, la cancelación y la imposibilidad de conocer el tamaño final.

Búsqueda semántica

Será necesario evaluar modelos de embeddings, umbrales y métricas sobre consultas representativas. La utilidad se medirá comparando resultados semánticos con búsquedas convencionales y verificando que los filtros de compatibilidad nunca se omiten.

Modelo local y privacidad

La interfaz desacoplada permitirá probar un modelo ejecutado localmente. Deberán compararse calidad, latencia, consumo energético, coste y privacidad frente al proveedor remoto. Los registros enviados al modelo se minimizarán y anonimizarán cuando sea posible.

Despliegue y escalado

Docker Compose cubrirá el entorno inicial. Un despliegue distribuido deberá incluir gestión de secretos, límites de recursos, copias de seguridad y observabilidad.

Accesibilidad e internacionalización

La interfaz podrá ampliarse con varios idiomas, navegación completa por teclado, mensajes compatibles con lectores de pantalla y pruebas sobre criterios WCAG. La accesibilidad deberá validarse desde las primeras pantallas y no añadirse únicamente al final.

7.4. Cierre

Batch Downloader propone reunir en una experiencia sencilla un conjunto complejo de responsabilidades: catálogo, seguridad, procesamiento

asíncrono, bundles y búsqueda semántica. El diseño establece límites claros para cada componente y proporciona una base verificable para comenzar la implementación sin presentar como terminadas funcionalidades que todavía deben construirse y probarse.

Bibliografía

- [1] Broadcom. Spring amqp reference documentation. <https://docs.spring.io/spring-amqp/reference/>, 2026. Consultado el 18 de junio de 2026.
- [2] Broadcom. Spring boot. <https://spring.io/projects/spring-boot/>, 2026. Consultado el 18 de junio de 2026.
- [3] Docker, Inc. Docker overview. <https://docs.docker.com/get-started/docker-overview/>, 2026. Consultado el 18 de junio de 2026.
- [4] Groq, Inc. Groq documentation: Overview. <https://console.groq.com/docs/overview>, 2026. Consultado el 18 de junio de 2026.
- [5] Andrew Katz and colaboradores. pgvector: Open-source vector similarity search for postgres. <https://github.com/pgvector/pgvector>, 2026. Consultado el 18 de junio de 2026.
- [6] Microsoft. Playwright for java: Installation and introduction. <https://playwright.dev/java/docs/intro>, 2026. Consultado el 18 de junio de 2026.
- [7] OpenAPI Initiative. Openapi specification. <https://spec.openapis.org/oas/latest.html>, 2025. Consultado el 18 de junio de 2026.
- [8] OpenJDK. Jdk 26. <https://openjdk.org/projects/jdk/26/>, 2026. Consultado el 18 de junio de 2026.
- [9] Patch My PC. Patch my pc home updater. <https://patchmypc.com/product/home-updater/>, 2026. Consultado el 20 de junio de 2026.
- [10] Secure By Design Inc. Ninite: Install or update multiple apps at once. <https://ninite.com/>, 2026. Consultado el 18 de junio de 2026.

- [11] Universidad de Burgos. Ubulabs. <https://www.ubu.es/servicio-de-informatica-y-comunicaciones/catalogo-de-servicios/software-tu-disposicion/ubulabs>, 2026. Portal de acceso: <https://ubulabs.ubu.es/login>. Consultado el 20 de junio de 2026.
- [12] VoidZero Inc. y colaboradores de Vite. Vite: Getting started. <https://vite.dev/guide/>, 2026. Consultado el 18 de junio de 2026.